

LABORATORIO #1
PARALELISMO E HILOS

ARSW

PRESENTADO A: DIEGO ANDRÉS TRIVIÑO GONZALEZ

REALIZADO POR:
MARIANA MALAGÓN
PAULA LOZANO

31/07//2026



UNIVERSIDAD

Parte I - Introducción a Hilos en Java

De acuerdo con lo revisado en las lecturas, complete las clases `CountThread`, para que las mismas definan el ciclo de vida de un hilo que imprima por pantalla los números entre A y B:

```
package edu.eci.arsw.threads;

/**
 *
 * @author Paula Lozano
 * @author Mariana Malagón
 */
public class CountThread implements Runnable{
    private int start, end;

    public CountThread(int start, int end){
        this.start = start;
        this.end = end;
    }

    @Override
    public void run(){
        for(int i = start; i <= end; i++){
            System.out.println(
                Thread.currentThread().getName() + ": " + i
            );
        }
    }
}
```

En esta clase llamada `CountThread` primero utilizará el método `run` de la interfaz `Runnable`, esto hace que se defina qué hace el hilo cuando corre, que en este caso, será imprimir los números dentro del rango dado.

Complete el método main de la clase CountMainThreads para que:
Cree 3 hilos de tipo CountThread, asignándole al primero el intervalo [0..99], al segundo [99..199], y al tercero [200..299].

```
package edu.eci.arsw.threads;
import java.util.Scanner;

/**
 *
 * @author Paula Lozano
 * @author Mariana Malagón
 */
public class CountThreadsMain {

    Run | Debug
    public static void main(String args[]){
        Scanner scanner = new Scanner(System.in);

        System.out.print(s: "Ingresa el valor de inicio: ");
        int start = scanner.nextInt();

        System.out.print(s: "Ingresa el valor final: ");
        int end = scanner.nextInt();

        scanner.close();

        if (start > end) {
            System.out.println(x: "El valor de inicio no puede ser mayor al valor final");
            return;
        }

        // First part
        executeThreads(start, end);
    }
}
```

Lo primero que hacemos en nuestra clase de CountMainThreads es pedirle al usuario que ingrese por consola cuáles son los números que quiere para su intervalo, el inicio y el final (verificando que el final sea mayor al inicio), y luego llamamos al método executeThreads que es el que arma y lanza los 3 hilos, esto pasándole el rango que ya validamos.

```

private static void executeThreads(int start, int end) {
    int intervalSize = (end - start + 1) / 3;

    int currentStart = start;

    for (int i = 0; i < 3; i++) {
        int currentEnd;
        if (i == 2) {
            currentEnd = end;
        } else {
            currentEnd = currentStart + intervalSize - 1;
        }

        Thread thread = new Thread(
            new CountThread(currentStart, currentEnd), // Task
            "Hilo " + (i+1) // Thread name
        );
        thread.start();

        currentStart = currentEnd + 1;
    }
}

```

El método executeThreads básicamente lo que hace es: primero divide el rango que le llegó en tres intervalos, entonces coge la cantidad total de números ($\text{end} - \text{start} + 1$) y los divide en 3 para saber aproximadamente cuántos números le toca a cada rango. Luego tenemos la variable currentStart que es la que va llevando el inicio del siguiente sub rango y se actualiza en cada vuelta del for, donde itera 3 veces, una por cada hilo.

Y la variable currentEnd que se calcula con currentStart y el tamaño del intervalo, a menos que ya sea la última iteración pues ahí ya es el número final.

Por último ya creamos el objeto Thread, cada uno se creará con su rango asignado y calculado previamente, con su respectivo nombre para ver mejor la diferencia al usar start() o al usar run() posteriormente.

Inicie los tres hilos con 'start()'.

```
thread.start();
```

Ejecute y revise la salida por pantalla.

```
Hilo 1: 0
Hilo 1: 1
Hilo 1: 2
Hilo 1: 3
Hilo 1: 4
Hilo 1: 5
Hilo 3: 200
Hilo 3: 201
Hilo 2: 100
Hilo 3: 202
Hilo 1: 6
Hilo 1: 7
Hilo 1: 8
Hilo 1: 9
Hilo 1: 10
Hilo 1: 11
Hilo 1: 12
Hilo 3: 203
Hilo 3: 204
Hilo 3: 205
Hilo 3: 206
Hilo 3: 207
Hilo 2: 101
Hilo 3: 208
Hilo 3: 209
Hilo 1: 13
Hilo 1: 14
Hilo 1: 15
Hilo 1: 16
Hilo 1: 17
Hilo 3: 210
Hilo 3: 277
Hilo 3: 278
Hilo 3: 279
Hilo 3: 280
Hilo 2: 147
Hilo 3: 281
Hilo 2: 148
Hilo 2: 149
Hilo 2: 150
Hilo 2: 151
Hilo 2: 152
Hilo 2: 153
Hilo 3: 282
Hilo 3: 283
Hilo 3: 284
Hilo 3: 285
Hilo 3: 286
Hilo 3: 287
Hilo 2: 154
Hilo 3: 288
Hilo 3: 289
Hilo 3: 290
Hilo 3: 291
Hilo 3: 292
Hilo 3: 293
Hilo 3: 294
Hilo 3: 295
Hilo 3: 296
Hilo 3: 297
Hilo 3: 298
```

Cambie el inicio con 'start()' por 'run()'. Cómo cambia la salida?, por qué?.

```
thread.run();
```

El resultado fue:

main: 0	main: 271
main: 1	main: 272
main: 2	main: 273
main: 3	main: 274
main: 4	main: 275
main: 5	main: 276
main: 6	main: 277
main: 7	main: 278
main: 8	main: 279
main: 9	main: 280
main: 10	main: 281
main: 11	main: 282
main: 12	main: 283
main: 13	main: 284
main: 14	main: 285
main: 15	main: 286
main: 16	main: 287
main: 17	main: 288
main: 18	main: 289
main: 19	main: 290
main: 20	main: 291
main: 21	main: 292
main: 22	main: 293
main: 23	main: 294
main: 24	main: 295
main: 25	main: 296
main: 26	main: 297
main: 27	main: 298
main: 28	main: 299
main: 29	

R/. Lo que nos dimos cuenta es que con start() sin se crean hilos de verdad, entonces los tres corren al tiempo y ya es el sistema operativo el que decide cuál va en cada momento, por eso es que en la salida se ven mezclados los tres hilos, y también esa salida cambia cada vez que ejecutábamos nuestro programa. En cambio, usando run() no se crea ningún hilo nuevo, es como si llamáramos al método directamente y se ejecuta todo dentro de main, uno detrás de otro, por eso sale ordenado y siempre dice main, así le hayamos asignado otro nombre al hilo.

Parte II - Ejercicio Black List Search

Para 'refactorizar' este código, y hacer que explote la capacidad multi-núcleo de la CPU del equipo, realice lo siguiente:

1. Cree una clase de tipo Thread que represente el ciclo de vida de un hilo que haga la búsqueda de un segmento del conjunto de servidores disponibles. Agregue a dicha clase un método que permita 'preguntarle' a las instancias del mismo (los hilos) cuantas ocurrencias de servidores maliciosos ha encontrado o encontró.

```
public class HostSearchThread extends Thread {
    private int start, end;
    private String ipAddress;
    private HostBlacklistsDataSourceFacade facade;
    private List<Integer> occurrences = new LinkedList<>();
    private int checkedListsCount = 0;

    public HostSearchThread(int start, int end, String ipAddress, HostBlacklistsDataSourceFacade facade){
        this.start = start;
        this.end = end;
        this.ipAddress = ipAddress;
        this.facade = facade;
    }

    @Override
    public void run() {
        for (int i = start; i <= end; i++) {
            if (facade.isInBlackListServer(i, ipAddress)) {
                occurrences.add(i);
                checkedListsCount++;
            }
        }
    }

    public int getOccurrencesCount() {
        return occurrences.size();
    }

    public List<Integer> getOccurrences() {
        return this.occurrences;
    }

    public int getCheckedListsCount() {
        return checkedListsCount;
    }
}
```

Creamos la clase HostSearchThread, en la que decidimos heredar de Thread directamente, en el método run es donde el hilo busca, recorre cada número de lista desde el inicio hasta el final, y por cada una le pregunta al facade que es el objeto que sabe consultar las listas negras, si esa lista tiene reportada la IP, si sí guarda el número en la lista occurrences y le suma 1 a checkedListCount.

2. Agregue al método 'checkHost' un parámetro entero N, correspondiente al número de hilos entre los que se va a realizar la búsqueda (recuerde tener en cuenta si N es par o impar!). Modifique el código de este método para que divida el espacio de búsqueda entre las N partes indicadas, y paralelice la búsqueda a través de N hilos. Haga que dicha función espere hasta que los N hilos terminen de resolver su respectivo sub-problema, agregue las ocurrencias encontradas por cada hilo a la lista que retorna el método, y entonces calcule (sumando el total de ocurrencias encontradas por cada hilo) si el número de ocurrencias es mayor o igual a BLACK_LIST_ALARM_COUNT. Si se da este caso, al final se DEBE reportar el host como confiable o no confiable, y

mostrar el listado con los números de las listas negras respectivas. Para lograr este comportamiento de 'espera' revise el método join del API de concurrencia de Java. Tenga también en cuenta:

- a. Dentro del método checkHost Se debe mantener el LOG que informa, antes de retornar el resultado, el número de listas negras revisadas VS. el número de listas negras total (línea 60). Se debe garantizar que dicha información sea verídica bajo el nuevo esquema de procesamiento en paralelo planteado.

```
public List<Integer> checkHost(String ipaddress, int N){

    LinkedList<Integer> blackListOccurrences=new LinkedList<>();

    HostBlacklistsDataSourceFacade skds=HostBlacklistsDataSourceFacade.getInstance();

    int totalServers = skds.getRegisteredServersCount();
    int interval = totalServers / N;

    HostSearchThread[] threads = new HostSearchThread[N];

    int start, end;

    for (int i = 0; i < N; i++) {
        start = i*interval;
        if (i == N-1) {
            end = totalServers-1;
        } else {
            end = (i+1)*interval - 1;
        }

        threads[i] = new HostSearchThread(start, end, ipaddress, skds);
        threads[i].start();
    }

    // Wait for all threads to finish
    for (int i = 0; i < N; i++) {
        try {
            threads[i].join();
        } catch (InterruptedException ex) {
            Thread.currentThread().interrupt();
        }
    }
}
```

```
    // Count total occurrences from all threads
    int checkedListsCount = 0;
    for (int i = 0; i < N; i++) {
        blackListOccurrences.addAll(threads[i].getOccurrences());
        checkedListsCount += threads[i].getCheckedListsCount();
    }

    int totalOccurrencesCount = blackListOccurrences.size();

    if (totalOccurrencesCount>=BLACK_LIST_ALARM_COUNT){
        skds.reportAsNotTrustworthy(ipaddress);
    }
    else{
        skds.reportAsTrustworthy(ipaddress);
    }

    LOG.log(Level.INFO, msg: "Checked Black Lists:{0} of {1}", new Object[]{checkedListsCount, skds.getRegisteredServersCount()});

    return blackListOccurrences;
}

private static final Logger LOG = Logger.getLogger(HostBlackListsValidator.class.getName());
```


Ahora el método checkHosts lo que hace es guardar en blackListOcurrrences todos los hallazgos de Ips desconfiables, las cuales se sacaron de totalServers los cuáles fueron revisados por N hilos. En el ciclo for es donde se reparten los rangos que cada hilo revisará, se les asigna un inicio, un final y la IP que se está buscando, luego se crea ese hilo y se inicia. El segundo for que tenemos es para esperar que todos hilos terminen antes de seguir y tener todo revisado para seguir. Ahora si se empieza a “recoger” lo que cada hilo encontró, para finalmente revisar si alguna IP estaba apareciendo 5 o más veces, lo que la hace desconfiable.

- b. Se sabe que el HOST 202.24.34.55 está reportado en listas negras de una forma más dispersa

```

15 public class Main {
16
17     Run | Debug
18     public static void main(String a[]){
19         HostBlackListsValidator hblv=new HostBlackListsValidator();
20         List<Integer> blackListOcurrrences=hblv.checkHost(ipaddress: "202.24.34.55", N: 10);
21         System.out.println("The host was found in the following blacklists:"+blackListOcurrrences);
22     }
23 }

```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL HISTORY TASK MONITOR

```

PS C:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS> c:\; cd 'c:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS'
deDetailsInExceptionMessages' '-cp' 'C:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS\target\classes' 'edu.eci.ar
ago. 02, 2026 10:42:23 P.ÁM. edu.eci.arsw.spamkeywordsdatasource.HostBlacklistsDataSourceFacade reportAsNotTrustworthy
INFORMACIÉN: HOST 202.24.34.55 Reported as NOT trustworthy
ago. 02, 2026 10:42:23 P.ÁM. edu.eci.arsw.blacklistvalidator.HostBlackListsValidator checkHost
INFORMACIÉN: Checked Black Lists:80.000 of 80.000
The host was found in the following blacklists:[29, 10034, 20200, 31000, 70500]

```

Efectivamente ese HOST es desconfiable, ya que se encontró en 5 listas negras.

- c. y que el host 212.24.24.55 NO está en ninguna lista negra.

```

17 public static void main(String a[]){
18     HostBlackListsValidator hblv=new HostBlackListsValidator();
19     List<Integer> blackListOcurrrences=hblv.checkHost(ipaddress: "212.24.24.55", N: 10);
20     System.out.println("The host was found in the following blacklists:"+blackListOcurrrences);
21 }
22
23

```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL HISTORY TASK MONITOR

```

PS C:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS> c:\; cd 'c:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS'
deDetailsInExceptionMessages' '-cp' 'C:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS\target\classes' 'edu.eci.a
ago. 02, 2026 10:56:01 P.ÁM. edu.eci.arsw.spamkeywordsdatasource.HostBlacklistsDataSourceFacade reportAsTrustworthy
INFORMACIÉN: HOST 212.24.24.55 Reported as trustworthy
ago. 02, 2026 10:56:01 P.ÁM. edu.eci.arsw.blacklistvalidator.HostBlackListsValidator checkHost
INFORMACIÉN: Checked Black Lists:80.000 of 80.000
The host was found in the following blacklists:[]
PS C:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS>

```

Efectivamente ese Host es confiable ya que no sale en ninguna lista negra.

Parte II.I Para discutir la próxima clase (NO para implementar aún)

La estrategia de paralelismo antes implementada es ineficiente en ciertos casos, pues la búsqueda se sigue realizando aún cuando los N hilos (en su conjunto) ya hayan encontrado el número mínimo de ocurrencias requeridas para reportar al servidor como malicioso. Cómo se podría modificar la implementación para minimizar el número de consultas en estos casos?, qué elemento nuevo traería esto al problema?

La implementación podría mejorarse utilizando un contador compartido entre todos los hilos, que registre el número total de veces que se ha encontrado la IP en las listas negras. Cada vez que un hilo encuentre una coincidencia, aumentaría este contador y, antes de continuar con la búsqueda, los hilos

podrían verificar si ya se alcanzaron las 5 ocurrencias necesarias. Si se alcanza este límite, podrían detenerse y así evitar consultas innecesarias.

Este cambio introduce un nuevo elemento al problema: los hilos ya no trabajarían de manera completamente independiente, sino que tendrían que compartir y modificar información entre ellos. Esto requiere controlar el acceso al contador para evitar que dos o más hilos intenten modificarlo al mismo tiempo y se produzcan resultados incorrectos.

Para manejarlo adecuadamente, se podrían utilizar mecanismos que ya conocemos, como `synchronized` o `lock`, que permiten controlar el acceso al contador compartido. De esta manera, se mantendría la ventaja del procesamiento en paralelo, pero se podría detener la búsqueda tan pronto como entre todos los hilos se alcance el número mínimo de ocurrencias.

Parte III - Evaluación de Desempeño

A partir de lo anterior, implemente la siguiente secuencia de experimentos para realizar la validación de direcciones IP dispersas (por ejemplo 202.24.34.55), tomando los tiempos de ejecución de los mismos (asegúrese de hacerlos en la misma máquina): Se usó un computador de 8 núcleos.

Para realizar este punto realizamos una pequeña modificación en Main, pusimos comentarios en las líneas que añadimos para este punto:

```
/**
 *
 * @author Mariana Malagón
 * @author Paula Lozano
 */
public class Main {

    Run | Debug
    public static void main(String a[]) {
        int cores = Runtime.getRuntime().availableProcessors(); // this is for point 3
        System.out.println("Núcleos disponibles: " + cores); // this is for point 3

        int N = 8; // this is for point 3

        HostBlackListsValidator hblv=new HostBlackListsValidator();

        long startTime = System.currentTimeMillis(); // this is for point 3
        List<Integer> blackListOcurrences=hblv.checkHost(ipaddress: "202.24.34.55", N);
        long endTime = System.currentTimeMillis(); // this is for point 3

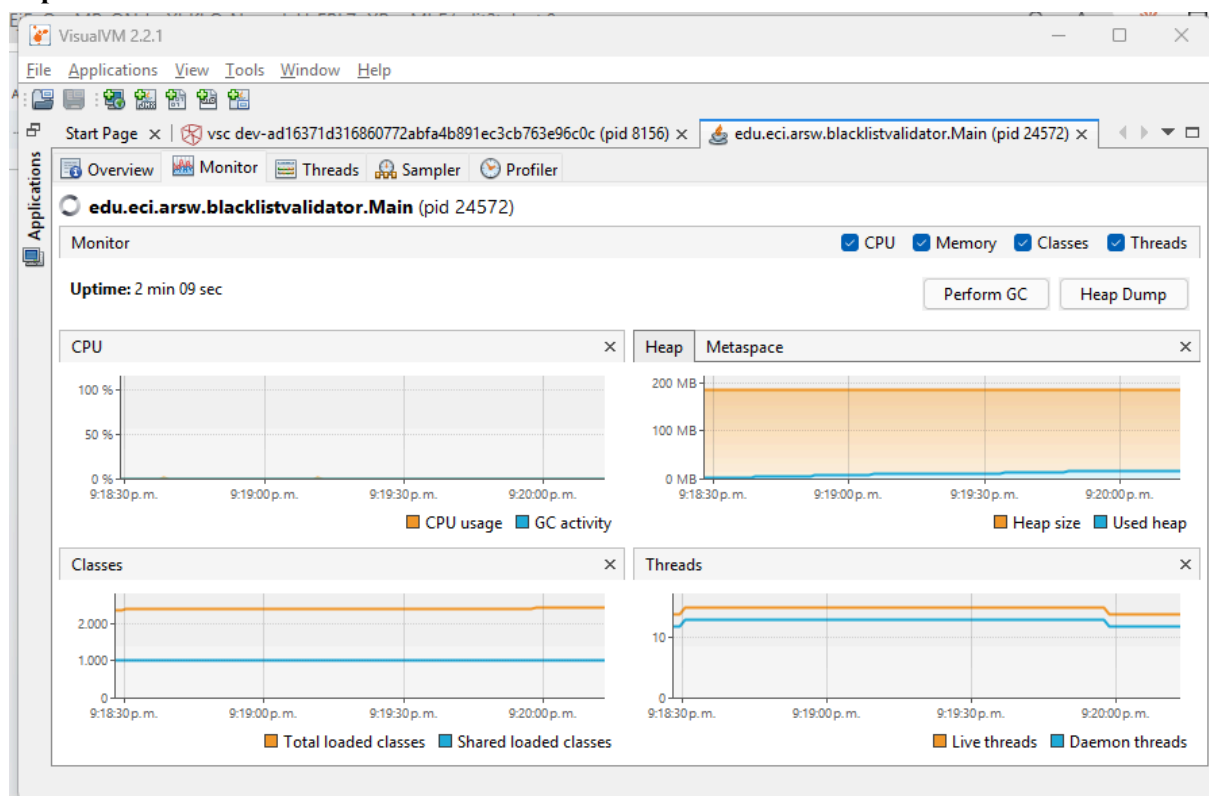
        System.out.println("The host was found in the following blacklists:"+blackListOcurrences);
        System.out.println("N=" + N + " hilos con tiempo: " + (endTime - startTime) + " ms"); // this is for point 3
    }
}
```

Se modificó el main para calcular automáticamente el número de núcleos con `Runtime.getRuntime().availableProcessors()`, extraer el número de hilos a una variable N fácil de cambiar entre corridas, y medir el tiempo de ejecución de `checkHost` con `System.currentTimeMillis()` antes y después de la llamada. Se usó la IP 202.24.34.55 por estar reportada de forma dispersa, lo que permite ver mejor las diferencias de tiempo entre las distintas cantidades de hilos.

Un solo hilo.

```
int N = 1; // this is for point 3
```

Captura de visualVM:



Respuesta en consola:

```
N=1 hilos con tiempo: 124239 ms
PS C:\Users\maria\Downloads\ARSM\ARSM_PARALLELISM_THREADS> c:; cd 'C:\Users\maria\Downloads\ARSM\ARSM_PARALLELISM_THREADS'; & 'C:\Program Files\Eclipse Adoptium\jdk-17.0.17.10-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\maria\Downloads\ARSM\ARSM_PARALLELISM_THREADS\target\classes' 'edu.eci.arsw.blacklistvalidator.Main'
Nucleos disponibles: 8
ago. 04, 2026 9:20:14 P.M. edu.eci.arsw.spamkeywordsdatasource.HostBlacklistsDataSourceFacade reportAsNotTrustworthy
INFORMACIN: HOST 202.24.34.55 Reported as NOT trustworthy
ago. 04, 2026 9:20:14 P.M. edu.eci.arsw.blacklistvalidator.HostBlacklistsValidator checkHost
INFORMACIN: Checked Black Lists:80.000 of 80.000
The host was found in the following blacklists:[29, 10034, 20200, 31000, 70500]
N=1 hilos con tiempo: 130545 ms
```

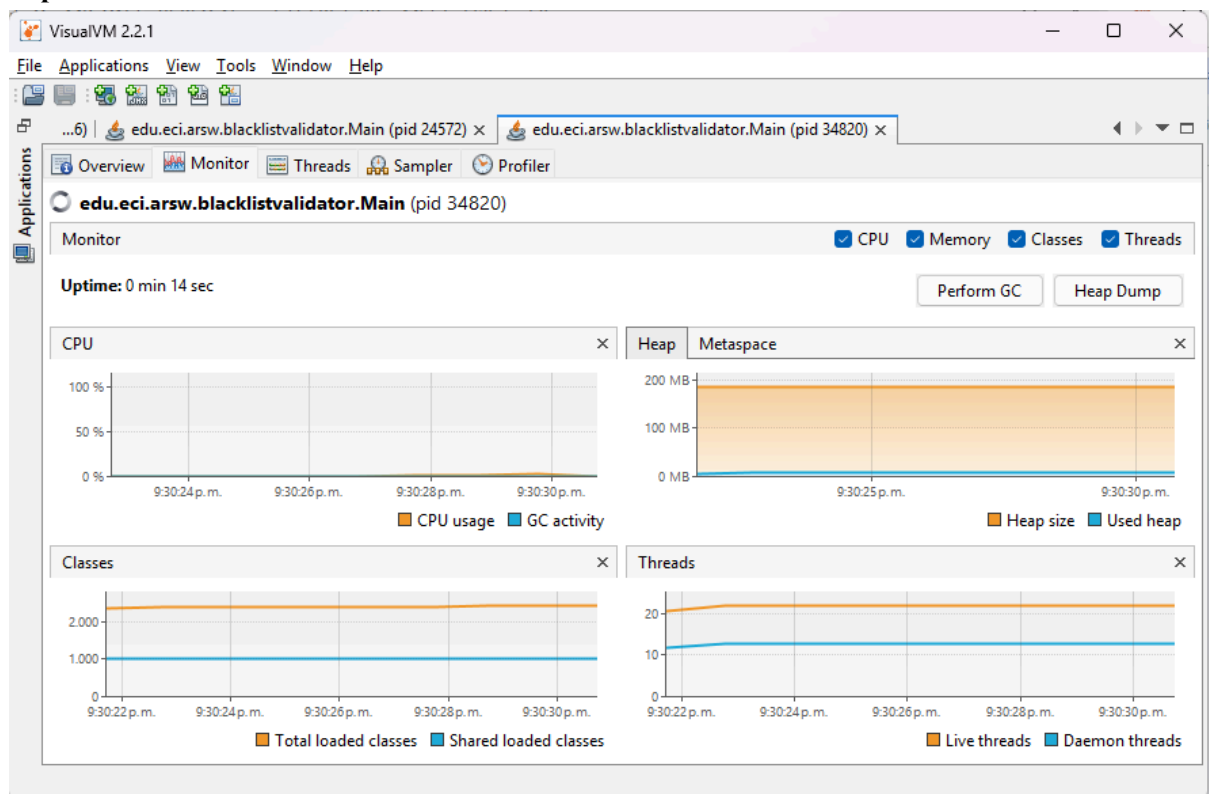
Conclusin:

Con N=1, el programa se demoró 2.45 min, la corrida más lenta de todas, porque las 80,000 consultas se hacen una tras otra sin ningún paralelismo. Aun así, la CPU se mantuvo casi en 0% durante toda la ejecución, lo que muestra que el tiempo se pierde esperando cada consulta, no calculando, evidenciando que se desaprovechan los demás núcleos del procesador. Esta corrida sirve como línea base para medir la mejora al agregar más hilos.

Tantos hilos como núcleos de procesamiento (haga que el programa determine esto haciendo uso del API Runtime).

```
int N = 8; // this is for point 3
```

Captura de visualVM:



Respuesta en consola:

```
PS C:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS> cd 'c:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS'; 8 'C:\Program Files\Eclipse Adoptium\jdk-17.0
.17-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS\target\classes' 'edu.eci.arsw.blackli
stvalidator.Main'
Núcleos disponibles: 8
ago. 04, 2026 9:30:31 P.ºM. edu.eci.arsw.spamkeywordsdatasource.HostBlacklistsDataSourceFacade reportAsNotTrustworthy
INFORMACIÓN: HOST 202.24.34.55 Reported as NOT trustworthy
ago. 04, 2026 9:30:31 P.ºM. edu.eci.arsw.blacklistvalidator.HostBlackListsValidator checkHost
INFORMACIÓN: Checked Black Lists:80.000 of 80.000
The host was found in the following blacklists:[29, 10034, 20200, 31000, 70500]
N=8 hilos con tiempo: 15112 ms
```

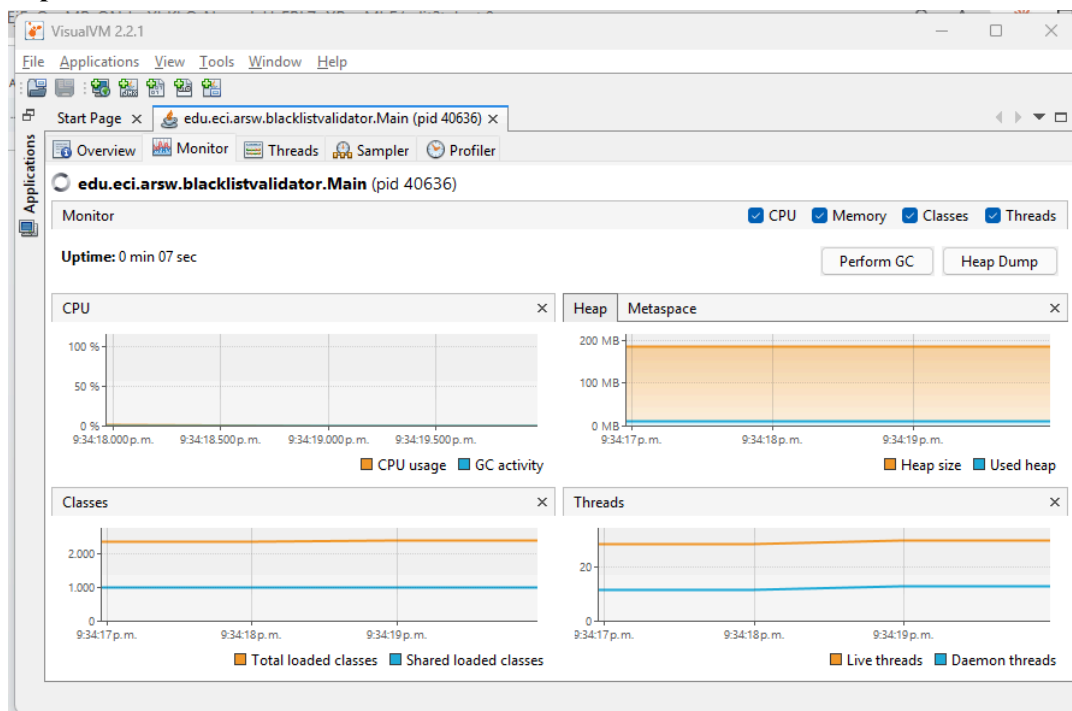
Conclusión:

Con N=8, el tiempo bajó a 15,112 ms, casi 10 veces más rápido que con 1 solo hilo. Sin embargo, en jVisualVM la CPU se mantuvo igual de baja que antes, casi sin subir lo que confirma que la mejora no vino de "usar más procesador", sino de poder tener 8 consultas esperando su respuesta al mismo tiempo en vez de una sola. Los threads vivos subieron a 15-17 (los 8 de búsqueda más los internos de la JVM), y la memoria se mantuvo estable, sin picos.

Tantos hilos como el doble de núcleos de procesamiento.

```
int N = 16; // this is for point 3
```

Captura de visualVM:



Respuesta en consola:

```
PS C:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS> c:: cd 'c:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS'; & 'C:\Program Files\Eclipse Adoptium\jdk-17.0
.17.10-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS\target\classes' 'edu.eci.arsw.blackli
stvalidator.Main'
Nucleos disponibles: 8
ago. 04, 2026 9:34:20 P.M. edu.eci.arsw.spamkeywordsdatasource.HostBlacklistsDataSourceFacade reportAsNotTrustworthy
INFORMACIÖN: HOST 202.24.34.55 Reported as NOT trustworthy
ago. 04, 2026 9:34:20 P.M. edu.eci.arsw.blacklistvalidator.HostBlackListsValidator checkHost
INFORMACIÖN: Checked Black Lists:80.000 of 80.000
The host was found in the following blacklists:[29, 10034, 20200, 31000, 70500]
N=16 hilos con tiempo: 7967 ms
```

Conclusión:

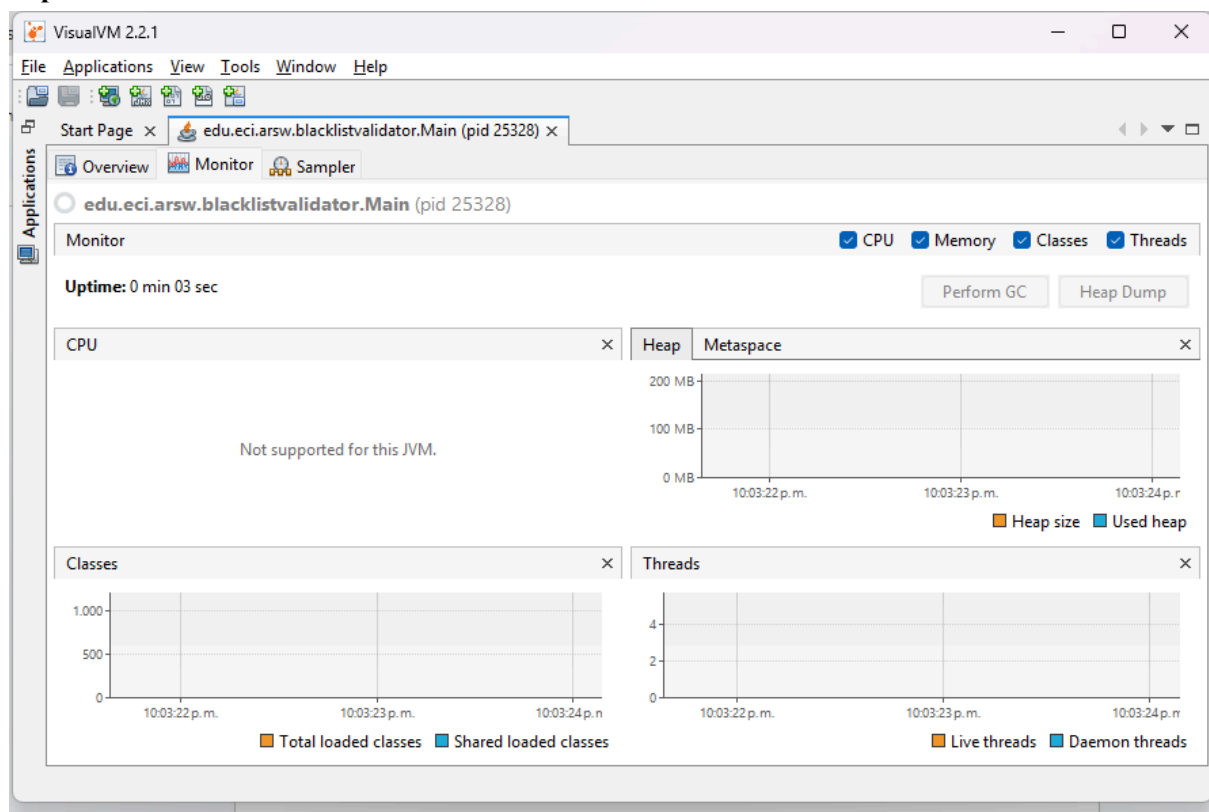
Con N=16, el tiempo bajó a 7,967 ms, prácticamente la mitad del tiempo que con N=8 (15,112 ms) y como 18 veces más rápido que con 1 solo hilo. Lo interesante es que la CPU sigue viéndose igual de baja en la gráfica, a pesar de tener el doble de hilos que núcleos físicos, refuerza la idea de que la

ganancia sigue viniendo de solapar más esperas al mismo tiempo, no de exigir más al procesador. Los threads vivos subieron a 20-25, y memoria/heap se mantuvieron estables como en las corridas anteriores.

50 hilos.

```
int N = 50; // this is for point 3
```

Captura de visualVM:



Respuesta en consola:

```
PS C:\Users\maria\Downloads\ARSM\PARALLELISM_THREADS> cd 'c:\Users\maria\Downloads\ARSM\PARALLELISM_THREADS'; & 'C:\Program Files\Eclipse Adoptium\jdk-17.0
.17.10-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\maria\Downloads\ARSM\PARALLELISM_THREADS\target\classes' 'edu.eci.arsw.blackli
stvalidator.Main'
Nucleos disponibles: 8
ago. 04, 2026 10:03:19 P.M. edu.eci.arsw.spamkeywordsdatasource.HostBlackListsDataSourceFacade reportAsNotTrustworthy
INFORMACIÓN: HOST 202.24.34.55 Reported as NOT trustworthy
ago. 04, 2026 10:03:20 P.M. edu.eci.arsw.blacklistvalidator.HostBlackListsValidator checkHost
INFORMACIÓN: Checked Black Lists:80.000 of 80.000
The host was found in the following blacklists:[29, 10034, 20200, 31000, 70500]
N=50 hilos con tiempo: 2857 ms
```

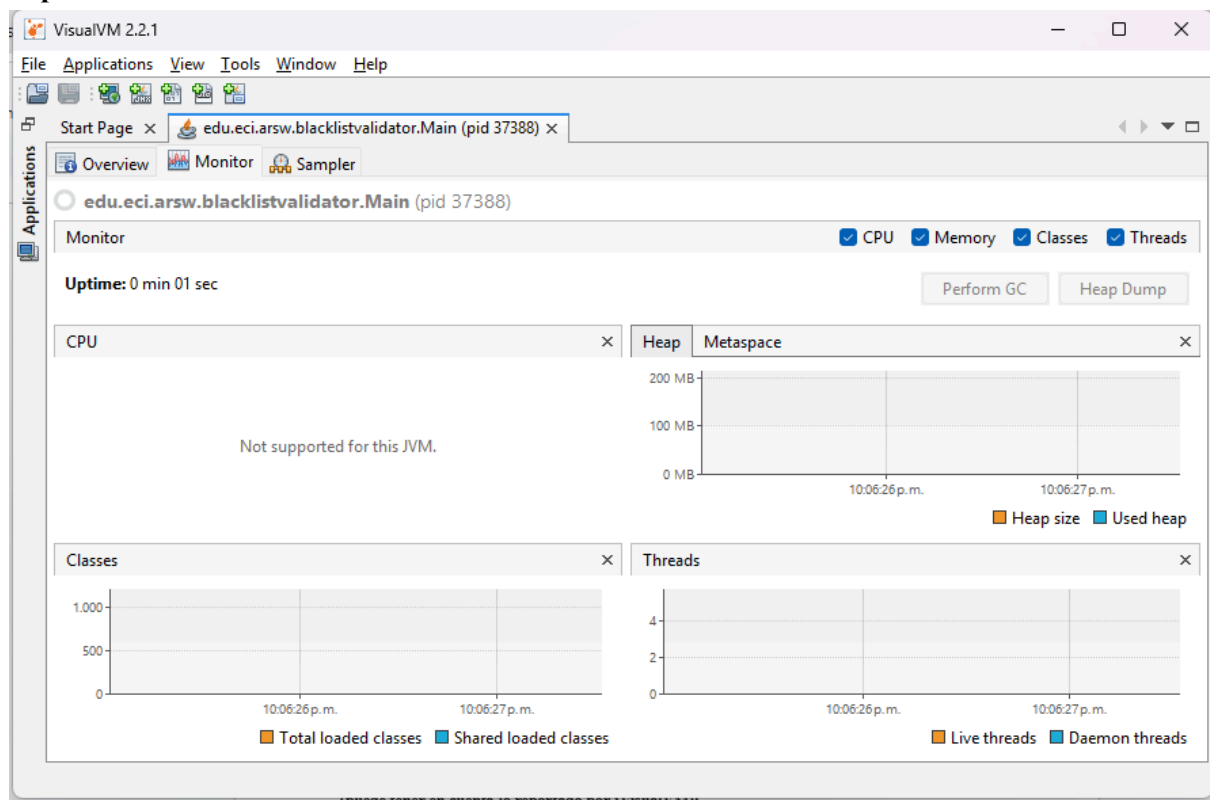
Conclusión:

Con N=50, el tiempo bajó a 2,857 ms, unas 51 veces más rápido que con 1 solo hilo. Sin embargo, jVisualVM no alcanzó a capturar datos útiles y las demás gráficas quedaron vacías, porque el proceso terminó en menos de 3 segundos, un tiempo demasiado corto para que el monitor logre muestrear algo. Esto muestra que, a partir de cierta cantidad de hilos, la ejecución se vuelve tan rápida que ya no es posible perfilarla visualmente con esta herramienta.

100 hilos.

```
int N = 100; // this is for point 3
```

Captura de visualVM:



Respuesta en consola:

```

PS C:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS> c++; cd 'c:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS\17.10-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\maria\Downloads\ARSW\ARSW_PARALLELISM_THREADS\17.10-hotspot\bin\java.exe' 'stvalidator.Main'
Nucleos disponibles: 8
ago. 04, 2026 10:06:23 P. M. edu.eci.arsw.spamkeywordsdatasource.HostBlacklistsDataSourceFacade reportAsNotTrustworthy
INFORMACI N: HOST 202.24.34.55 Reported as NOT trustworthy
ago. 04, 2026 10:06:23 P. M. edu.eci.arsw.blacklistvalidator.HostBlacklistsValidator checkHost
INFORMACI N: Checked Black Lists:80.000 of 80.000
The host was found in the following blacklists:[29, 10034, 20200, 31000, 70500]
N=100 hilos con tiempo: 1379 ms

```

Conclusi n:

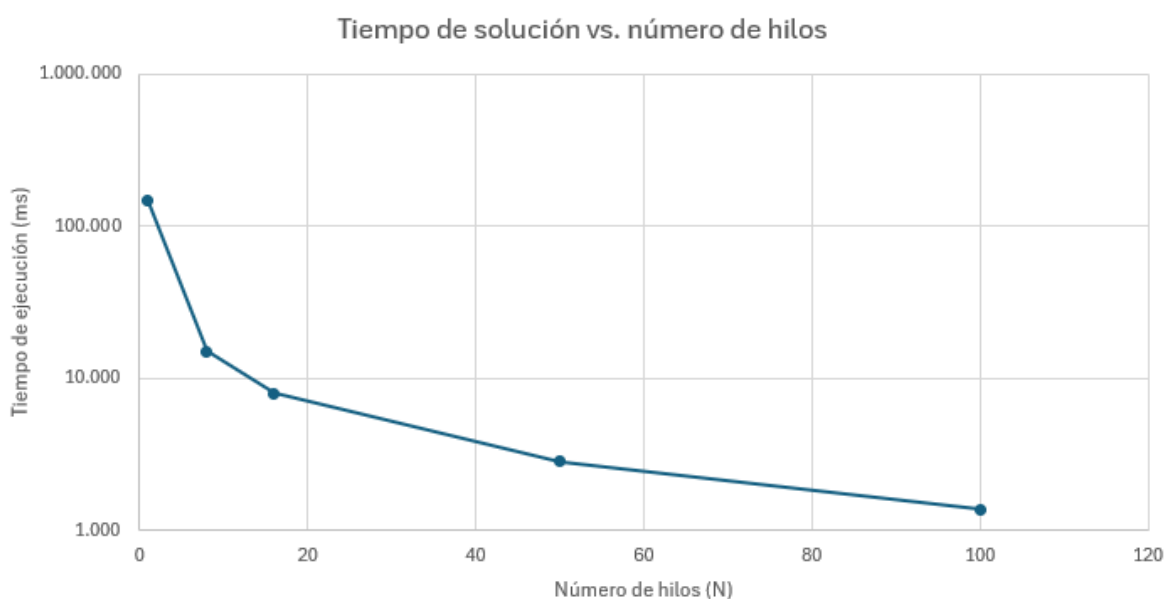
Con N=100, el tiempo baj  a 1,379 ms, la corrida m s r pida de todas , casi 107 veces m s r pido que con 1 solo hilo. Igual que con N=50, jVisualVM no logr  capturar nada, esta vez con apenas 1 segundo de vida del proceso. Esto refuerza la misma conclusi n: con cantidades muy altas de hilos, la ejecuci n es tan corta que ninguna herramienta de monitoreo alcanza a engancharse a tiempo para mostrar datos reales.

Con lo anterior, y con los tiempos de ejecuci n dados, haga una gr fica de tiempo de soluci n vs. n mero de hilos. Analice y plantee hip tesis con su compa ero para las siguientes preguntas (puede tener en cuenta lo reportado por jVisualVM):

Usamos los siguientes datos para generar la gr fica:

N (hilos)	Tiempo (ms)
1	147.218
8	15.112
16	7.967
50	2.857
100	1.379

La gr fica:



Hip tesis final:

Con los 5 resultados se ve clar simo que a m s hilos, menos tiempo, pero la mejora no es infinita ni proporcional para siempre: de 1 a 8 hilos el tiempo cae much simo (de 147 segundos a 15), pero de 50

a 100 ya la diferencia es mucho menor en términos relativos, o sea que la ganancia se va aplanando. Además, con jVisualVM vimos que la CPU casi nunca sube, ni siquiera con 16 hilos corriendo, lo que sugiere que el programa no está limitado por el procesador sino por la cantidad de consultas que se hacen una por una (o en grupos), por eso paralelizar ayuda tanto aunque uno tenga solo 8 núcleos. La hipótesis es que si se siguiera aumentando el número de hilos más allá de 100, en algún punto el tiempo dejaría de bajar y hasta podría empezar a subir un poco, porque ya no habría más "espera" que solapar y sí más hilos peleándose por los mismos 8 núcleos.

Parte IV - Ejercicio Black List Search

1. Según la ley de Amdahls:

$$S(n) = \frac{1}{(1 - P) + \frac{P}{n}}$$

, donde $S(n)$ es el mejoramiento teórico del desempeño, P la fracción paralelizable del algoritmo, y n el número de hilos. A mayor n , mayor debería ser dicha mejora. ¿Por qué el mejor desempeño no se logra con los 500 hilos?, ¿cómo se compara este desempeño cuando se usan 200?

La Ley de Amdahl indica que aumentar el número de hilos puede mejorar el desempeño de la parte del algoritmo que se puede ejecutar en paralelo. Sin embargo, esto no significa que entre más hilos se creen, mejor será siempre el resultado.

Por ejemplo, si un computador tiene 8 núcleos y se crean 500 hilos, no es posible ejecutar los 500 al mismo tiempo. Los núcleos tendrán que ir alternando entre los diferentes hilos, lo cual también requiere tiempo y trabajo por parte del computador. Por esta razón, llega un momento en el que crear más hilos deja de generar una mejora importante e incluso puede hacer que el programa tarde más.

Al utilizar 200 hilos ocurre algo similar, porque siguen siendo muchos más hilos que núcleos, pero el computador tiene una menor cantidad de hilos que administrar. Por esta razón, en las pruebas 200 hilos pueden presentar un mejor desempeño que 500. Esto demuestra que aumentar el número de hilos no garantiza necesariamente un mejor rendimiento.

2. ¿Cómo se comporta la solución usando tantos hilos de procesamiento como núcleos comparado con el resultado de usar el doble de éste?

Si el computador tiene K núcleos y utilizamos K hilos, idealmente cada núcleo podría encargarse de ejecutar uno de ellos al mismo tiempo. Esto permite aprovechar de una manera adecuada la capacidad de procesamiento disponible.

Si aumentamos la cantidad a $2K$ hilos, el computador sigue teniendo solamente K núcleos. Por lo tanto, los nuevos hilos tendrán que compartir esos mismos núcleos y alternarse durante su ejecución.

La Ley de Amdahl establece que:

$$s(K) = \frac{1}{(1-P) + \frac{P}{K}}$$

Al aumentar n , el mejoramiento teórico aumenta, pero cada vez en menor proporción debido a la parte del algoritmo que no puede ejecutarse en paralelo. Además, en la práctica estamos limitados por la cantidad de núcleos disponibles.

Por esto, utilizar el doble de hilos que de núcleos no significa obtener el doble de rendimiento. El resultado puede ser parecido, ligeramente mejor o incluso peor, dependiendo del trabajo que esté realizando el computador.

3. De acuerdo con lo anterior, si para este problema en lugar de 100 hilos en una sola CPU se pudiera usar 1 hilo en cada una de 100 máquinas hipotéticas, ¿la ley de Amdahls se aplicaría mejor?

En una sola máquina, 100 hilos tienen que compartir los núcleos disponibles. En cambio, si se tuviera un hilo en cada una de 100 máquinas, habría muchos más recursos disponibles para ejecutar las diferentes partes de la búsqueda al mismo tiempo.

Por esta razón, el comportamiento podría acercarse más a la mejora planteada teóricamente por la Ley de Amdahl, ya que existirían más recursos reales para ejecutar en paralelo. Sin embargo, seguiría existiendo el límite generado por la parte del algoritmo que no puede paralelizarse.

Además, al utilizar varias máquinas sería necesario intercambiar información entre ellas para reunir los resultados, lo que también requiere tiempo. Por lo tanto, la mejora real no sería exactamente igual a la planteada de manera teórica.

Si en lugar de esto se usaran c hilos en $100/c$ máquinas distribuidas (siendo c es el número de núcleos de dichas máquinas), ¿se mejoraría?. Explique su respuesta.

Si cada máquina tiene c núcleos y se utilizan c hilos en cada una, se estarían aprovechando todos sus núcleos. Al utilizar $100/c$ máquinas seguiríamos teniendo en total 100 hilos:

$$c \times \frac{100}{c} = 100$$

Por ejemplo, si cada máquina tuviera 4 núcleos, podrían utilizarse 4 hilos en 25 máquinas, manteniendo los 100 hilos en total.

Esta opción permitiría aprovechar los diferentes núcleos de cada máquina sin necesitar 100 máquinas distintas. Podría ser una buena alternativa porque mantiene una alta capacidad de procesamiento utilizando menos máquinas. Sin embargo, el resultado final seguiría dependiendo de cuánto del algoritmo se pueda ejecutar en paralelo y del tiempo necesario para compartir y reunir los resultados entre las diferentes máquinas.

Referencias:

colaboradores de Wikipedia. (2024, 31 julio). Speedup. Wikipedia, la Enciclopedia Libre
<https://es.wikipedia.org/wiki/Speedup>

Oracle. (2025). *Thread (Java SE 25 & JDK 25)*. Java Platform, Standard Edition & Java Development Kit Documentation.
<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Thread.html>